# Binary Search

**ALGORITHM:**

arr = [2, 4, 6, 9, 11, 12, 14, 20, 36, 48] (Sorted Array)

Ascending order

target = 36

1. Find the middle element
2. Check if target > middle ⇒ search in right
   else ⇒ search in left
3. Check if target = middle ⇒ Found the target element.

**EXAMPLE:** In the above array,

arr = [2, 4, 6, 9, 11, 12, 14, 20, 36, 48] target = 36

mid = $\frac{\text{start} + \text{end}}{2}$ ⇒ $\frac{0+9}{2}$ ⇒ 4 [ The element at index 4 is the middle element = 11 ]

**Check:** Is '36' > '11' ⇒ Yes! → Check on right side

**Now,** arr = [2, 4, 6, 9, 11, 12, 14, $\boxed{20}$, 36, 48]

5 6 7 8 9

**Check,** Is '36' > '20' ⇒ Yes! → Check on right side

**Now,** mid = $\frac{8+9}{2}$ = 8 → The element at 8 is 36 → Element found

**Here, the space in which we are searching is getting divided into two spaces**

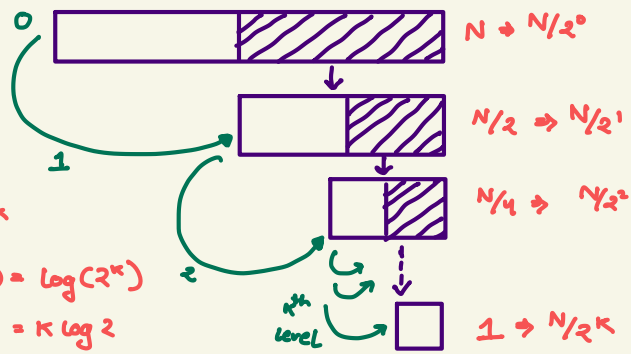
⇒ **TIME COMPLEXITY:**

Best Case : O(1)

Worst Case : O(log n)

**Explanation:**

Find the maximum number of comparisons.




$\frac{N}{2^k} = 1$

$N = 2^k$

$\log(N) = \log(2^k)$

$\log(N) = k \log 2$

$k = \log N / \log 2$

$\boxed{k = \log_2 N}$

TOTAL NUMS. OF COMPARISON IN WORST CASE

SIZE OF ARRAY

However, there is a problem if we use "(start + end)/2" in a program. If start and end values are too large, adding them may exceed the maximum integer value. Hence, this may cause overflow.

Therefore, "start + ((end-start)/2)" should be used.

$$mid = start + \frac{(end - start)}{2}$$

$$= \frac{2start + end - start}{2}$$

$$= \frac{start + end}{2}$$

[ It yields the same results and eliminates the possibility of overflow ]

## Order agnostic Binary Search

If we recieve a sorted array in descending order, we can just reverse the logic we used for an ascending array. However, how do we know the type of array we have?

Order-Agnostic Binary search helps in identifying what kind of array we have and performing the required operations on it.

**Here**, start > end → Descending Order

end > start → Ascending Order